JAVASCRIPT

L'HÉRITAGE

JAVASCRIPT

L'héritage

Supposons avoir une classe `Date`

```
class Date {
    constructor(year, month, day){
        this.year = year;
        this.month = month;
        this.day = day;
    }
    getDate(){
        return this.day+"."+this.month+"."+this.year;
    }
}

const date = new Date(2026, 10, 15);
console.log(date.getDate()); // 15.10.2026
```

On souhaite maintenant créer une deuxième classe `DateTime`, stockant en plus l'heure. Il serait bien de pouvoir réutiliser la classe `Date`, sans devoir réimplémenter tout son contenu.

JAVASCRIPT

L'héritage

JavaScript permet de le faire de la manière suivante

```
class DateTime extends Date {    // Le mot clé "extends" indique l'héritage
    constructor(year, month, day, hours, minutes, seconds){
        super(year, month, day) // "super" est le constructeur de la classe parent
        this.hours = hours;
        this.minutes = minutes;
        this.seconds = seconds;
    }
    getTime(){
        return this.hours+":"+this.minutes+":"+this.seconds;
    }
}

const dateTime = new DateTime(2023, 12, 23, 12, 30, 45);
console.log(dateTime.getDate()); // 23.12.2023  Les fonctions parent sont accessibles
console.log(dateTime.getTime()); // 12:30:45
```

JAVASCRIPT

L'héritage

Supposons avoir une méthode `print` dans la classe Date.

```
class Date {
    constructor(year, month, day){
        this.year = year;
        this.month = month;
        this.day = day;
    }
    getDate(){
        return this.day+"."+this.month+"."+this.year;
    }
    print(){
        console.log("Nous sommes le "+this.getDate());
    }
}

const date = new Date(2026, 10, 15);
date.print(); // Nous sommes le 15.10.2026
```

Il va falloir modifier cette méthode dans la classe enfant DateTime.

JAVASCRIPT

L'héritage

On peut écraser (*override*) des méthodes venues des classes parent.

```
class DateTime extends Date {    // Le mot clé "extends" indique l'héritage
    constructor(year, month, day, hours, minutes, seconds){
        super(year, month, day) // "super" est le constructeur de la classe parent
        this.hours = hours;
        this.minutes = minutes;
        this.seconds = seconds;
    }
    getTime(){
        return this.hours+":"+this.minutes+":"+this.seconds;
    }
    print(){
        console.log("Nous sommes le "+this.getDate()+" à "+this.getTime());
    }
}
const dateTime = new DateTime(2023, 12, 23, 12, 30, 45);
dateTime.print(); // Nous sommes le 23.12.2023 à 12:30:45
```

JAVASCRIPT

L'héritage

Pour utiliser spécifiquement une fonction de la classe parent, on utilise aussi `super`

```
class DateTime extends Date {
    constructor(year, month, day, hours, minutes, seconds){ /* ... */ }
    getTime(){
        return this.hours+":"+this.minutes+":"+this.seconds;
    }
    print(){
        console.log("Nous sommes le "+this.getDate()+" à "+this.getTime());
    }
    printDate(){
        // Ici on doit utiliser super, car si on avait utilisé this.print(),
        // la fonction ci-dessus aurait été appelée (à cause de l'override)
        super.print();
    }
}
const dateTime = new DateTime(2023, 12, 23, 12, 30, 45);
dateTime.printDate(); // Nous sommes le 23.12.2023
```

JAVASCRIPT

L'héritage

Comment choisir entre héritage et attribut ?

X sera un attribut de Y si on peut dire *"Un Y **possède** un X"*

X sera un parent de Y si on peut dire *"Un Y **est** un X"*

```
class Motor{
    constructor(){/*...*/}
    start(){/*...*/}
    stop(){/*...*/}
}
class Car extends Motor {
    constructor(){/*...*/}
    break(){/*...*/}
}
```

```
class Motor{
    constructor(){/*...*/}
    start(){/*...*/}
    stop(){/*...*/}
}
class Car {
    constructor(){
        this.motor = new Motor();
        /*...*/
    }
    break(){/*...*/}
}
```

L'idée de droite est la bonne : *Une voiture possède un moteur*, une voiture n'est *pas* un moteur

JAVASCRIPT

L'héritage

Comment choisir entre héritage et attribut ?

X sera un attribut de Y si on peut dire *"Un Y **possède** un X"*

X sera un parent de Y si on peut dire *"Un Y **est** un X"*

```
class Shape{
    constructor(){/*...*/}
    setColor(){/*...*/}
}
class Disk{
    constructor(){
        this.shape = new Shape();
    }
    area(){/*...*/}
}
```

```
class Shape{
    constructor(){/*...*/}
    setColor(){/*...*/}
}
class Disk extends Shape{
    constructor(){/*...*/}
    area(){/*...*/}
}
```

L'idée de droite est la bonne : *Un cercle est un type de forme*, un cercle ne possède *pas* une forme (la forme n'est pas un composant, un attribut du cercle)